

Spring Data JPA

Services, Query Methods y Consultas Avanzadas

Prof. Alejandro Peñaranda

Universidad ICESI – Compunet II

- 1 Capa de Servicios y Repositorios
- 2 Query Methods por Convención
- 3 Consultas Avanzadas con @Query
- 4 Estudio Autónomo: Transacciones y Paginación

Sección 01

Capa de Servicios

Invocación de Repositorios y Separación de Capas

- **Aislamiento de Lógica:** El Controller jamás accede directamente a la base de datos.
- **Orquestación:** El Service coordina múltiples repositorios y entidades.
- **Reutilización:** Reglas consumibles por REST, procesos batch o eventos.

Principio SRP

El Service encapsula las reglas de negocio y delega las operaciones de acceso a datos al Repositorio.

Controller (Presentación)

- Mapea rutas y verbos HTTP.
- Recibe `@RequestParam` / `@PathVariable`.
- Valida tipos de datos de entrada.
- Retorna `ResponseEntity<T>`.

Service (Negocio)

- Aplica las políticas del dominio.
- Invoca Query Methods del repositorio.
- Transforma entidades a DTOs.
- Orquesta múltiples entidades.

- **Inyección por Constructor:** Garantiza inmutabilidad con atributos `final`.
- **Lombok `@RequiredArgsConstructor`:** Genera el constructor de dependencias automáticamente.
- **Mocks y Tests:** Permite instanciar el Service en tests aislando los Repositorios con Mockito.

Inmutabilidad

Evitar `@Autowired` sobre campos para eliminar referencias nulas y facilitar pruebas unitarias.

```
@Service
@RequiredArgsConstructor
public class CursoService {
    private final CursoRepository cursoRepository;

    // Invoca un metodo basico de busqueda en el repositorio
    public List<Curso> listarCursosPorDepartamento(String depto) {
        return cursoRepository.findByDepartamento(depto);
    }

    // Invoca la busqueda por identificador unico
    public Optional<Curso> obtenerCursoPorId(Long id) {
        return cursoRepository.findById(id);
    }
}
```

Sección 02

Query Methods

Introspección y Ejercicios Granulares

¿Qué es un Query Method?

- **Definición:** Consulta derivada declarada únicamente mediante la firma de un método en un Repository.
- **Convención de Nombres:** El nombre del método expresa directamente el criterio de búsqueda sin escribir SQL.
- **Cero Boilerplate:** Elimina la necesidad de escribir implementaciones repetitivas de persistencia.

Idea Clave

El nombre del método Java se convierte en la propia sentencia de persistencia ejecutada contra la base de datos.

- **Introspección en Arranque:** Spring analiza el nombre de cada método de la interfaz.
- **Traducción Automática:** Convierte nombres Java en cláusulas SQL/JPQL.
- **Cero Implementación:** Tú sólo declaras la firma; Spring genera el proxy.

Sin SQL Manual

El desarrollador no escribe consultas manuales para el 80 % de las búsquedas del dominio.

Búsqueda Única: findBy... con Optional<T>

- **Propósito:** Consultar un registro único mediante un campo identificador o único.
- **Retorno Optional<T>:** Modela explícitamente la presencia o ausencia de la entidad.
- **Evita NullPointerException:** El Service decide si desempaquetar o lanzar una excepción controlada.

Claves Únicas

Ideal para buscar por correo institucional, cédula o nombres únicos en el sistema.

Ejemplo 1: Búsqueda Única con Optional<T>

```
// 1. Declaracion en UsuarioRepository
@Repository
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByCorreoInstitucional(String correo);
}

// 2. Invocacion limpia desde UsuarioService
@Service
@RequiredArgsConstructor
public class UsuarioService {
    private final UsuarioRepository usuarioRepository;

    public Optional<Usuario> obtenerPorCorreo(String correo) {
        return usuarioRepository.findByCorreoInstitucional(correo);
    }
}
```

Comprobación de Existencia: `existsBy...` con `boolean`

- **Propósito:** Verificar rápidamente si ya existe un registro que cumpla el criterio.
- **Retorno boolean:** Devuelve `true` si existe al menos una coincidencia; si no, `false`.
- **Ultra-eficiente:** Genera `SELECT COUNT(id) > 0`, sin traer entidades pesadas a memoria.

Rendimiento

Ideal para validaciones previas de disponibilidad de correos o códigos antes de registrar.

Ejemplo 2: Verificación de Existencia con boolean

```
// 1. Declaracion en UsuarioRepository
@Repository
public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    boolean existsByCorreoInstitucional(String correo);
}

// 2. Invocacion en UsuarioService para validaciones livianas
@Service
@RequiredArgsConstructor
public class UsuarioService {
    private final UsuarioRepository usuarioRepository;

    public boolean existeCorreo(String correo) {
        return usuarioRepository.existsByCorreoInstitucional(correo);
    }
}
```

- **Propósito:** Recuperar todas las entidades que coincidan con un criterio común.
- **Retorno `List<T>`:** Contiene todas las coincidencias ordenadas o en bruto.
- **Colección Vacía:** Si no hay registros coincidentes, Spring retorna una lista vacía (nunca `null`).

Múltiples Filas

Ideal para filtrar entidades que comparten una categoría, departamento o estado común.

Ejemplo 3: Búsqueda de Colecciones con `List<T>`

```
// 1. Declaracion en CursoRepository
@Repository
public interface CursoRepository extends JpaRepository<Curso, Long> {
    List<Curso> findByDepartamento(String departamento);
}

// 2. Invocacion en CursoService
@Service
@RequiredArgsConstructor
public class CursoService {
    private final CursoRepository cursoRepository;

    public List<Curso> listarPorDepartamento(String depto) {
        return cursoRepository.findByDepartamento(depto);
    }
}
```

- **Propósito:** Conocer el total exacto de registros que cumplen una condición.
- **Retorno long:** Devuelve el número escalar directamente desde el motor SQL.
- **Cero Sobrecarga:** Evita el antipatrón de hacer `findAll().size()` que colapsa la memoria JVM.

Métricas

Genera directamente `SELECT COUNT(*)` en base de datos de manera atómica y veloz.

Ejemplo 4: Conteo Numérico con countBy y long

```
// 1. Declaracion en CursoRepository
@Repository
public interface CursoRepository extends JpaRepository<Curso, Long> {
    long countByDepartamento(String departamento);
}

// 2. Invocacion en CursoService
@Service
@RequiredArgsConstructor
public class CursoService {
    private final CursoRepository cursoRepository;

    public long contarCursosPorDepartamento(String depto) {
        return cursoRepository.countByDepartamento(depto);
    }
}
```

- **Operador And:** Exige que se cumplan múltiples condiciones simultáneas en el WHERE.
- **Operador Or:** Permite coincidencia de cualquiera de las condiciones especificadas.
- **Banderas Booleanas:** Sufijos directos como `ActiveTrue` o `ActiveFalse`.

Lógica Booleana

Permite componer filtros complejos concatenando nombres de atributos de forma declarativa.

Ejemplo 5: Filtro Combinado con And y ActiveTrue

```
// 1. Declaracion en ProfesorRepository
@Repository
public interface ProfesorRepository extends JpaRepository<Profesor, Long> {
    List<Profesor> findByDepartamentoAndActiveTrue(String depto);
}

// 2. Invocacion en ProfesorService
@Service
@RequiredArgsConstructor
public class ProfesorService {
    private final ProfesorRepository profesorRepository;

    public List<Profesor> listarProfesoresActivos(String depto) {
        return profesorRepository.findByDepartamentoAndActiveTrue(depto);
    }
}
```

- **Between:** Rango cerrado inclusivo (campo BETWEEN min AND max).
- **GreaterThan / GreaterThanEqual:** Valores mayores o mayores e iguales que un umbral.
- **LessThan / LessThanEqual:** Valores menores o menores e iguales que un límite.

Filtros de Escala

Ideal para consultar rangos de créditos académicos, edades, fechas o presupuestos.

Ejemplo 6: Rangos de Créditos con Between

```
// 1. Declaracion en CursoRepository
@Repository
public interface CursoRepository extends JpaRepository<Curso, Long> {
    List<Curso> findByCreditosBetween(int minCreditos, int maxCreditos);
    List<Curso> findByCreditosGreaterThanOrEqual(int creditosMinimos);
}

// 2. Invocacion en CursoService
@Service
@RequiredArgsConstructor
public class CursoService {
    private final CursoRepository cursoRepository;

    public List<Curso> listarPorRangoCreditos(int min, int max) {
        return cursoRepository.findByCreditosBetween(min, max);
    }
}
```

- **Containing:** Traduce a `LIKE %texto %` para búsqueda de fragmentos o palabras clave.
- **IgnoreCase:** Aplica `UPPER()` en SQL para comparaciones insensibles a mayúsculas.
- **OrderBy...Asc / Desc:** Aplica cláusula `ORDER BY` directamente en la base de datos.

Buscadores Web

Los parámetros se procesan vía `PreparedStatement`, previniendo vulnerabilidades de inyección SQL.

Ejemplo 7: Búsqueda de Texto y Ordenamiento

```
// 1. Declaraciones en repositorios
public interface CursoRepository extends JpaRepository<Curso, Long> {
    List<Curso> findByNombreContainingIgnoreCase(String fragmento);
}

public interface ProfesorRepository extends JpaRepository<Profesor, Long> {
    List<Profesor> findByEspecialidadIgnoreCaseOrderByApellidoAsc(String esp);
}

// 2. Invocacion en Services correspondientes
public List<Curso> buscarCursosPorTexto(String texto) {
    return cursoRepository.findByNombreContainingIgnoreCase(texto);
}

public List<Profesor> listarPorEspecialidadOrdenados(String esp) {
    return profesorRepository.findByEspecialidadIgnoreCaseOrderByApellidoAsc(esp);
}
```

- **Guion Bajo (-):** Permite descender a través de relaciones JPA sin ambigüedad.
- **Relaciones @ManyToOne:**
`findByProfesor_Id(Long id)` o por atributos anidados.
- **Relaciones @ManyToMany:**
`findByRoles_Nombre(String rol).`
- **Tablas Intermedias:**
`existsById_EstudianteId(Long estudianteId).`

JOIN Automático

Spring Data detecta las relaciones mapeadas y genera automáticamente el INNER JOIN relacional.

Ejemplo 8: Navegación de Relaciones con Guion Bajo (-)

```
// 1. Repositorios navegando relaciones ManyToOne y ManyToMany
public interface CursoRepository extends JpaRepository<Curso, Long> {
    List<Curso> findByProfesor_Id(Long profesorId);
    List<Curso> findByProfesor_Departamento(String depto);
}

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    List<Usuario> findByRoles_Nombre(String nombreRol);
}

// 2. Invocacion en UsuarioService
public List<Usuario> listarUsuariosPorRol(String rol) {
    return usuarioRepository.findByRoles_Nombre(rol);
}
```

Sección 03

Consultas con @Query

JPQL, Native SQL y Modificaciones Masivas

¿Cuándo los Query Methods no son Suficientes?

- **Métodos Kilométricos:** Nombres excesivamente largos y difíciles de mantener.
- **Agregaciones Complejas:** Cláusulas COUNT, AVG, GROUP BY, HAVING.
- **JOINS Explícitos:** Consultas que cruzan tablas intermedias asociativas.

Control Total

La anotación @Query permite definir exactamente la consulta requerida.

JPQL (Java Persistence Query Language)

- Basado en clases y atributos Java.
- 100 % portable entre motores SQL.
- Valida sintaxis al arrancar la app.
- Mapea entidades directamente.

Native Query (SQL del Motor)

- Basado en tablas y columnas físicas.
- Acoplado al motor (PostgreSQL/H2).
- Permite funciones específicas y CTEs.
- Se valida al momento de ejecutarse.

- **Parámetros Nombrados:** Se vinculan usando `:nombre` y la anotación `@Param("nombre")`.
- **Sentencias UPDATE/DELETE:** Deben marcarse obligatoriamente con `@Modifying`.
- **Retorno de Filas:** Retornan un valor primitivo `int` con el total de filas modificadas.
- **Contexto Transaccional:** Operaciones de modificación requieren transacción activa.

Actualización Masiva

`@Modifying` actualiza directamente en la base de datos sin cargar entidades a memoria.

Ejemplo 9: Consulta con JPQL y Native SQL

```
@Repository
public interface EstudianteRepository extends JpaRepository<Estudiante, Long> {

    // 1. JPQL: Orientado al modelo de entidades (EstudianteCurso)
    @Query("SELECT ec.estudiante FROM EstudianteCurso ec " +
        "WHERE ec.curso.id = :cursoId AND ec.estudiante.active = true " +
        "ORDER BY ec.estudiante.apellido ASC")
    List<Estudiante> buscarEstudiantesPorCursoJPQL(@Param("cursoId") Long cursoId);

    // 2. Native Query: SQL directo con INNER JOIN sobre tablas fisicas
    @Query(value = "SELECT e.* FROM estudiante e " +
        "INNER JOIN estudiante_curso ec ON e.id = ec.estudiante_id " +
        "WHERE ec.curso_id = :cursoId AND e.active = true " +
        "ORDER BY e.apellido ASC", nativeQuery = true)
    List<Estudiante> buscarEstudiantesPorCursoNativo(@Param("cursoId") Long cursoId);
}
```

04

Estudio Autónomo

Transacciones ACID, Rollback y Paginación

ACID

Garantía de Integridad

“O se ejecutan todas las operaciones con éxito, o ninguna deja huella en la base de datos.”

- **Proxy Dinámico (AOP):** Spring intercepta el método y gestiona la conexión JDBC.
- **Delimitación Atómica:** Abre transacción al entrar y confirma con `commit`.
- **Gestión de Fallos:** Si ocurre una excepción no controlada, revierte con `rollback`.

Consistencia

Evita estados intermedios inconsistentes entre múltiples tablas ante fallos imprevistos.

Comportamiento de Rollback

- **Por defecto:** Rollback en `RuntimeException` y `Error`.
- **Checked Exceptions:** Por defecto NO provocan rollback.
- **Regla robusta:** Usar `@Transactional(rollbackFor = Exception.class)`.

Optimización readOnly

- Con `readOnly = true`:
- Hibernate apaga el *Dirty Checking*.
- Ahorra CPU y memoria Heap.
- Permite enrutar a réplicas de lectura.

```
@Service
@RequiredArgsConstructor
public class MatriculaService {
    private final EstudianteRepository estudianteRepo;
    private final CursoRepository cursoRepo;
    private final EstudianteCursoRepository estudianteCursoRepo;

    @Transactional(rollbackFor = Exception.class)
    public EstudianteCurso matricular(Long estId, Long cursoId) {
        var est = estudianteRepo.findById(estId)
            .orElseThrow(() -> new IllegalArgumentException("Estudiante no existe"));
        var cur = cursoRepo.findById(cursoId)
            .orElseThrow(() -> new IllegalArgumentException("Curso no existe"));

        var mat = estudianteCursoRepo.save(new EstudianteCurso(est, cur));

        // Regla tardia: si excede creditos, lanza excepcion y dispara ROLLBACK
        if (cur.getCreditos() > 4) {
            throw new RuntimeException("Creditos excedidos -> ROLLBACK de la matricula");
        }
        return mat;
    }
}
```

Paginación: ¿Por qué Evitar findAll()?

- **Saturación de Memoria:** Cargar miles de filas satura el Heap (OutOfMemory).
- **Latencia de Red:** Transferir payloads JSON gigantescos bloquea al cliente.
- **Carga en BD:** Bloqueos prolongados de lectura y alto I/O en disco.

Escalabilidad

La paginación fragmenta grandes volúmenes de datos entregando bloques controlados bajo demanda.

Page<T> (Paginado Completo)

- Ejecuta `SELECT ... LIMIT` y `COUNT(*)`.
- Conoce el total de páginas y registros.
- Ideal para interfaces web con barra de páginas (1, 2, 3...).

Slice<T> (Scroll Infinito)

- Ejecuta `SELECT ... LIMIT + 1`.
- Sólo sabe si existe página siguiente.
- Ahorra el costo de contar todo con `COUNT(*)`.
- Ideal para aplicaciones móviles y feeds infinitos.

```
// Repositorio: Spring Data inyecta LIMIT y OFFSET segun el Pageable
public interface CursoRepository extends JpaRepository<Curso, Long> {
    Page<Curso> findByDepartamento(String depto, Pageable pageable);
}

// Servicio: construye el PageRequest con paginacion y orden
@Transactional(readOnly = true)
public Page<Curso> listarPaginados(String depto, int page, int size, String sort) {
    Pageable pageable = PageRequest.of(page, size, Sort.by(sort).ascending());
    return cursoRepository.findByDepartamento(depto, pageable);
}

// Controlador REST: expone los parametros de consulta al cliente
@GetMapping("/paginados")
public ResponseEntity<Page<Curso>> getPaginados(
    @RequestParam String depto, @RequestParam(defaultValue = "0") int page,
    @RequestParam(defaultValue = "10") int size, @RequestParam(defaultValue = "nombre") String sort)
{
    return ResponseEntity.ok(cursoService.listarPaginados(depto, page, size, sort));
}
```

100%

Arquitectura Profesional

“El diseño riguroso de Query Methods y @Query en la persistencia, coordinado por Servicios limpios, garantiza un backend expresivo, desacoplado y escalable.”